PART 5 · 2H

Supabase로 회원·데이터 관리하기

Auth · Database · Storage · RLS

Claude Code로 만드는 나만의 GitHub 사이트

화면은 Cloudflare·Vercel에 두고, 로그인·데이터·파일은 Supabase에 맡긴다.
데이터를 지키는 것은 키가 아니라 RLS 정책이라는 점을 사례로 확인한다.

이번 파트에서 가져갈 세 가지

별도 서버 없이 회원제 사이트를 만드는 구조

1

화면과 백엔드를 나눈다

사이트(HTML·JS)는 배포 플랫폼에, 회원·데이터·파일은 Supabase에 둔다.

2

데이터를 지키는 것은 RLS다

행 수준 보안(RLS)이 꺼져 있으면 공개 키만으로 누구나 전체 데이터를 읽는다.

3

키는 두 종류, 공개 가능한 것은 하나뿐이다

publishable 키는 화면에 넣어도 되고, secret 키는 절대 공개하지 않는다.

Supabase = 오픈소스 Postgres 데이터베이스에 인증·파일 저장을 묶은 서비스

회원제 사이트에는 화면 뒤에서 일하는 백엔드가 필요하다

기본 지식 · 프런트엔드(front-end)와 백엔드(back-end)

프런트엔드 = 화면

- 사용자가 보고 누르는 부분
- HTML·CSS·JS, 배포 플랫폼에 둔다
- Part 3·4에서 이미 만들었다

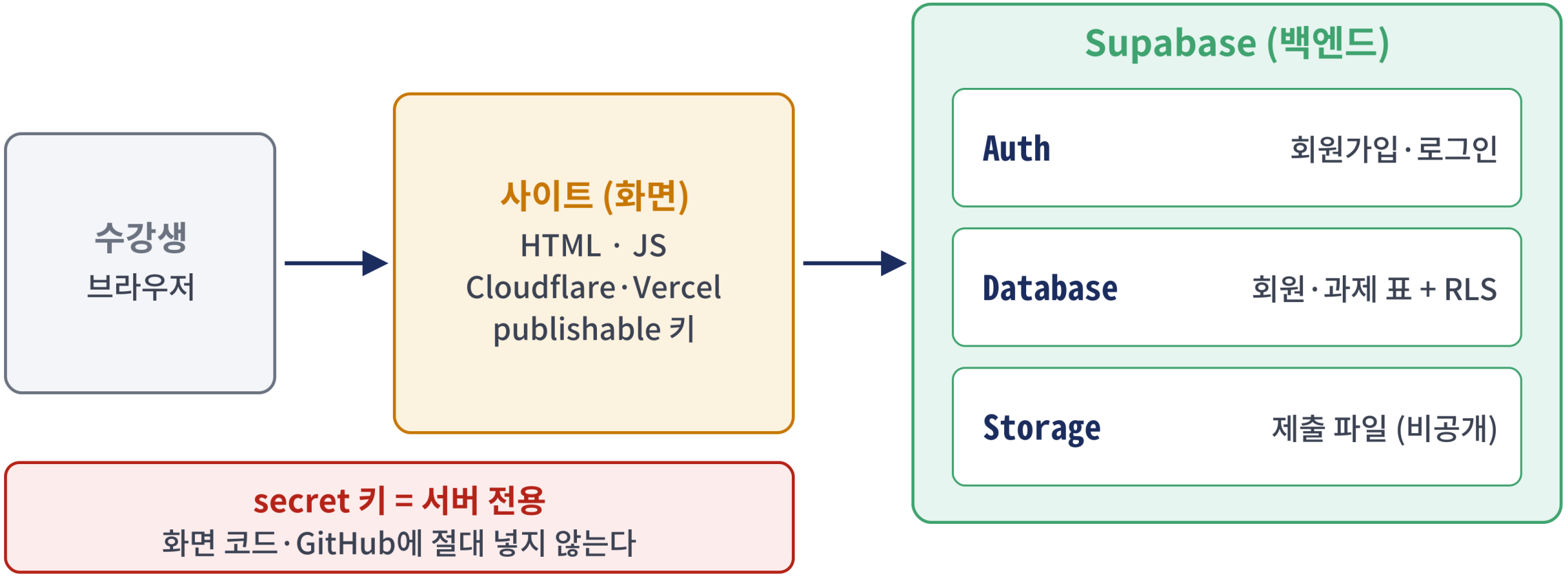
백엔드 = 뒤편

- 누가 로그인했는지 확인한다
- 데이터를 저장하고 꺼내 준다
- Supabase가 대신 맡는다

예전에는 백엔드를 직접 만들고 서버를 운영해야 했다. 지금은 서비스로 빌려 쓴다

사이트는 publishable 키로 Supabase에 요청한다

수강생 브라우저 → 사이트(화면) → Supabase(Auth·Database·Storage)



Supabase의 세 기능이 회원·과제·파일을 나눠 맡는다

구성 · 이 과정의 과제 제출 사이트에서 각 기능이 하는 일

기능	하는 일	이 과정에서
Auth (인증)	회원가입·로그인·이메일 확인	수강생·강사 계정
Database (Postgres)	표 형태로 데이터 저장·조회	회원 정보, 과제 제출 기록
Storage (저장소)	파일 보관 (공개·비공개 버킷)	과제 첨부 파일 (비공개)

버킷(bucket)

- Storage 안에서 파일을 담는 최상위 폴더. 비공개 버킷은 권한 있는 사람만 연다

RLS는 같은 표에서도 사람마다 보이는 행을 다르게 한다

기본 지식 · RLS(Row Level Security, 행 수준 보안)

RLS = '이 행은 누가 읽고 쓸 수 있는가'를 표마다 정책(policy)으로 적어 두는 규칙

RLS를 켜고 정책을 걸면

- 로그인한 본인 행만 보인다
- 관리자만 전체를 본다
- 정책이 없으면 아무것도 안 보인다

RLS를 끄면

- 공개 키만으로 전체 행이 보인다
- 로그인하지 않아도 조회된다
- 수정·삭제까지 열릴 수 있다

같은 질의라도 로그인한 사람에 따라 결과가 달라진다

정책: 본인 행(user_id = auth.uid()) 또는 관리자(is_admin())만 조회 허용

submissions 표 (전체)

id	작성자	과제
1	김A	Day1 실습
2	이B	Day1 실습
3	김A	Day2 실습

정책 (policy)

```
user_id = auth.uid()
  or is_admin()
```

김A로 로그인해 전체 조회

1번 · 3번 행만 돌아온다

2번(이B)은 보이지 않는다

같은 질의, 다른 결과

관리자로 로그인해 전체 조회

1 · 2 · 3번 모두 돌아온다

is_admin() 이 참이기 때문

RLS가 꺼져 있으면 로그인하지 않은 사람에게도 세 행이 모두 보인다

화면에 넣어도 되는 키는 publishable 키 하나뿐이다

API 키 체계 · 기존 anon·service_role 키는 2026년 말 폐기 예정 (Supabase Docs)

키	형식	넣는 곳	권한
publishable	sb_publishable_...	화면 코드 (공개 가능)	RLS가 허용한 만큼
secret	sb_secret_...	서버 전용 (공개 금지)	RLS 무시 (전체 권한)
anon (구)	eyJ...	화면 코드	publishable과 같음
service_role (구)	eyJ...	서버 전용	secret과 같음

publishable 키가 안전한 이유는 키가 아니라 RLS에 있다 — RLS가 없으면 공개 키가 곧 열쇠다

RLS 누락 하나로 170개 서비스의 데이터가 열렸다

사례 · Lovable CVE-2025-48757 — AI로 만든 앱들의 Supabase 설정 결함

2025.3
보안 연구자가 플랫폼에 제보

170개
RLS가 부실했던 프로젝트

303개
로그인 없이 조회된 엔드포인트

2025.5
CVE 공개 (제보 후 약 2개월)

원인

- 화면 코드의 공개 키로 직접 조회
- 정책이 없거나 너무 느슨했다

교훈

- 정책이 '있는지'보다 '막는지' 시험
- 계정 둘·비로그인으로 교차 확인

프로젝트 생성은 대시보드에서 2분이면 끝난다

실습 ① · supabase.com, 무료 플랜 기준

가입

supabase.com

GitHub 계정으로 가입한다

새 프로젝트

New project

이름을 정한다 (영문)

DB 비밀번호

Database password

생성 후 안전한 곳에 보관한다

리전

Region: Seoul

한국 수강생이면 서울을 고른다

프로젝트 ID = 대시보드 주소의 project/ 뒤 문자열. MCP 연결에 쓴다

MCP를 연결하면 Claude Code가 DB를 직접 다룬다

실습 ② · MCP(Model Context Protocol) = AI가 외부 서비스를 쓰게 하는 연결 규격

[입력] 명령어 · 터미널 (프로젝트 폴더에서)

```
claude mcp add --scope project --transport http supabase "https://mcp.supabase.com/mcp?project_ref=프로젝트ID"
```

[입력] 명령어 · claude 실행 후 입력

```
/mcp  →  supabase  →  Authenticate (브라우저 로그인)
```

안전 수칙 (Supabase 공식 권장)

- project_ref로 한 프로젝트만 연결, 설정 뒤에는 &read_only=true 추가
- 운영 중인 실제 DB에는 연결하지 않고, 도구 실행은 매번 승인한다

테이블과 보안 정책은 한 번의 지시로 함께 만든다

실습 ③ · 표를 만들 때 RLS를 같이 요구하는 것이 핵심이다

[입력] 프롬프트 · Claude Code

```
profiles(이름·역할),
submissions(과제명·파일·제출일)
테이블을 만들고,
본인 데이터만 읽고 쓸 수 있게
RLS 정책을 걸어줘.
역할이 admin이면 전체를
볼 수 있게 해줘.
```

[출력] Claude가 생성 · SQL 일부

```
alter table submissions
  enable row level security;

create policy "본인 또는 관리자"
  on submissions for select
  to authenticated
  using (user_id = auth.uid()
         or is_admin());
```

RLS를 요구하지 않으면 표는 만들어지지만 아무도 보호받지 못한다

무료 플랜은 과정 기간에 쓰기에 충분하다

제약 · 2026년 기준 무료 플랜 한도 (최신 값은 supabase.com/pricing 확인)

항목	무료 한도
프로젝트	조직당 2개
데이터베이스	프로젝트당 500MB
파일 저장	프로젝트당 1GB
월간 활성 사용자	5만 명
미사용 시	1주일 활동이 없으면 일시정지 → 대시보드에서 복구

가입 확인 메일은 기본 발송량이 적다. 수강생이 많으면 이메일 확인을 끄거나 자체 SMTP를 연결한다

회원 데이터는 최소한만 받고 과정이 끝나면 지운다

운영 원칙 · 수강생 개인정보를 다루는 사이트의 기본

받는 것

- 이름·이메일 (제출 확인용)
- 과제 파일 (비공개 버킷)
- 가입 화면에 수집 항목 한 줄 안내

받지 않는 것

- 주민번호·전화번호·주소
- 회사 내부 자료가 든 파일
- 과정과 무관한 설문 정보

과정 종료 후: Authentication → Users에서 회원 삭제, 버킷 비우기

구조와 보안 원리를 설명할 수 있는지 점검한다

점검 · 키와 RLS의 관계가 핵심이다

Q1	publishable 키가 화면 코드에 들어가도 괜찮은 이유는	**키**
Q2	secret 키가 GitHub에 올라가면 어떤 일이 생기는가	**위험**
Q3	Lovable 사례에서 '정책이 있다'와 '막는다'는 어떻게 다른가	**사례**
Q4	MCP에 project_ref와 read_only를 붙이는 이유는	**안전**

구조를 이해했으니 실제 사이트를 완성한다

종합 · Part 6에서 템플릿으로 과제 제출 사이트를 만든다

1

화면 + Supabase = 서버 없는 회원제 사이트

Auth가 로그인을, Database가 기록을, Storage가 파일을 맡는다.

2

RLS가 데이터를 지킨다

표를 만들면 RLS를 켜고 정책을 건다. 그리고 막히는지 직접 시험한다.

3

publishable은 공개, secret은 금지

구 키(anon·service_role)는 2026년 말 폐기 예정이다. 새 키 체계로 시작한다.

다음 · Part 6 실습 프로젝트 — 과정 사이트 완성과 배포